

İSTANBUL GEDİK ÜNİVERSİTESİ

Bilgisayar Mühendisliği Bölümü

GestureGlobe

El Hareketleriyle 3D Dünya Küresi Kontrolü

Gerçek Zamanlı Derin Öğrenme Tabanlı Gesture Tanıma Sistemi

Öğrenci:

Utku Mert Geçgel

Ders:

İleri Derin Öğrenme

Öğretim Üyesi:

Dr. Mehmet Fatih Yüce

İstanbul Gedik Üniversitesi

26 Mayıs 2026

Özet

Bu çalışmada, gerçek zamanlı el hareketi tanınması kullanılarak web tarayıcısında 3 boyutlu bir dünya küresini kontrol eden uçtan uca bir derin öğrenme sistemi geliştirilmiştir. Sistem; veri toplama, model eğitimi ve web entegrasyonu olmak üzere üç ana aşamadan oluşmaktadır. MediaPipe Hands kütüphanesi ile elde edilen 21 el landmarkı, ön işleme adımından geçtikten sonra üç ayrı derin öğrenme mimarisine — Çok Katmanlı Algılayıcı (MLP), 1 Boyutlu Evrişimsel Sinir Ağı (1D-CNN) ve Uzun Kısa Süreli Bellek (LSTM) — beslenmiştir. 386 örnekli bir veri kümesi üzerinde gerçekleştirilen deneylerde, **1D-CNN modeli %100 test doğruluğuyla** en iyi performansı sergilemiştir. Eğitilen model, ONNX Runtime Web aracılığıyla tarayıcıda çalıştırılmış ve Three.js tabanlı 3 boyutlu sahne ile entegre edilmiştir. Uygulama; açık avuç, kısırtma ve yumruk hareketlerini algılamasının yanı sıra küreyi döndürmek, yakınlaştırmak ve kilitlemek için bu hareketleri gerçek zamanlı olarak kullanabilmektedir.

İçindekiler

1	Giriş	3
1.1	Motivasyon	3
1.2	Proje Kapsamındaki Temel Katkılar	3
2	İlgili Çalışmalar	3
3	Sistem Mimarisi	4
3.1	Ön İşleme Adımı	4
4	Veri Kümesi	4
4.1	Veri Toplama	4
4.2	Veri Kümesi İstatistikleri	5
5	Model Mimarileri	5
5.1	Çok Katmanlı Algılayıcı (MLP)	5
5.2	1 Boyutlu Evrişimsel Sinir Ağı (1D-CNN)	6
5.3	Uzun Kısa Süreli Bellek (LSTM)	6
5.4	Eğitim Konfigürasyonu	6
6	DeneySEL Sonuçlar	7
6.1	Model Karşılaştırması	7
6.2	Sınıf Bazlı Performans (1D-CNN)	7
6.3	Neden 1D-CNN Başarılı?	7
7	Web Uygulaması	7
7.1	Mimari Bileşenler	7
7.2	ONNX Runtime Entegrasyonu	8
7.3	Gesture – Eylem Eşleştirmesi	9
7.4	Kullanılan Teknolojiler	9

8	Tartışma	9
8.1	Başarılı Yönler	9
8.2	Sınırlılıklar	9
8.3	Gelecek Çalışmalar	10
9	Sonuç	10
10	Ek Çalışmalar	10

1. Giriş

İnsan-bilgisayar etkileşimi (HCI) alanında, klavye ve fare gibi geleneksel giriş aygıtlarının yerini almaya aday olan yöntemlerden biri de el hareketi tanınmasıdır. Bilgisayar görü ve derin öğrenme alanlarındaki hızlı gelişim sayesinde, kameradan gerçek zamanlı el hareketi tanıma sistemleri artık yüksek doğrulukta çalışabilmekte ve pek çok uygulama alanı bulmaktadır.

Bu projede “GestureGlobe” adlı, el hareketleriyle kontrol edilebilen interaktif bir 3 boyutlu dünya küresi uygulaması geliştirilmiştir. Projenin temel amacı, derin öğrenme tabanlı bir gesture tanıma modelini uçtan uca tasarlamak; veri toplama, model eğitimi, model ihracatı ve tarayıcı entegrasyonu aşamalarını bir arada ele almaktır.

1.1. Motivasyon

Dokunmaksızın bir sistemi kontrol etme fikri; sağlık, eğitim ve sanal gerçeklik gibi pek çok alanda büyük değer taşımaktadır. Web teknolojilerinin olgunlaşması ile birlikte, ONNX Runtime Web ve WebGL gibi araçlar sayesinde karmaşık derin öğrenme modellerinin yalnızca tarayıcı üzerinde çalıştırılması mümkün hale gelmiştir. Bu proje, bahsedilen teknolojileri birleştirerek pratik ve demo düzeyde çalışan bir sistem ortaya koymaktadır.

1.2. Proje Kapsamındaki Temel Katkıları

- El landmarklarından elde edilen özelliklerle özel bir veri kümesi oluşturulmuştur.
- Üç farklı derin öğrenme mimarisi eğitilmiş ve karşılaştırılmıştır.
- En iyi model (1D-CNN), ONNX formatıyla tarayıcıya aktarılmıştır.
- Gerçek zamanlı gesture tanıma, 3 boyutlu sahne kontrolüyle entegre edilmiştir.

2. İlgili Çalışmalar

El hareketi tanıma literatüründe MediaPipe [1], elin 21 landmarkını milisaniyeler içinde tespit edebilen hafif ve hassas bir çözüm olarak öne çıkmaktadır. Bu çalışmada MediaPipe, ham görüntü işleme yükünü tamamen üstlenmekte; modelin görevi ise yalnızca bu landmark koordinatlarını sınıflandırmaktan ibarettir.

Landmark tabanlı gesture tanıma yaklaşımında, görüntü piksellerinin doğrudan kullanıldığı CNN modellerine kıyasla çok daha az parametre yeterli olmakta ve model kameraya olan uzaklık ile aydınlatma koşullarından bağımsız hale gelmektedir [2]. 1D-CNN mimarisinin landmark dizileri üzerinde etkili olduğu çeşitli çalışmalarla gösterilmiştir [3].

ONNX (Open Neural Network Exchange) standardı, PyTorch’ta eğitilen modellerin tarayıcı ortamında ONNX Runtime Web aracılığıyla verimli şekilde çalıştırılmasını sağlamaktadır [4]. Bu yaklaşım, sunucu gerektirmeyen, istemci tarafında koşan bir yapay zeka deneyimine olanak tanımaktadır.

3. Sistem Mimarisi

Sistem, birbiriyle zincirlenen dört katmandan oluşmaktadır:

1. **Görüntü Yakalama:** Web kamerası görüntüsü her karede MediaPipe Hands'e iletilir.
2. **Landmark Tespiti:** MediaPipe, elde 21 adet 3 boyutlu nokta (x, y, z koordinatları) çıkarır.
3. **Ön İşleme ve Sınıflandırma:** Landmarklar normalize edildikten ve mesafeler hesaplandıktan sonra ONNX modelinde sınıflandırma yapılır.
4. **Sahne Kontrolü:** Sınıflandırma sonucu, Three.js 3 boyutlu sahnesine iletilerek küre kontrol edilir.

3.1. Ön İşleme Adımı

Ham landmark koordinatları modele beslenmeden önce iki adımlı bir ön işlemeden geçirilmektedir:

Adım 1 — Normalizasyon: Bilek noktası (landmark 0) orijin olarak alınıp tüm koordinatlardan çıkarılır. Ardından, oluşturulan vektörün maksimum mutlak değere bölünmesiyle koordinatlar $[-1, 1]$ aralığına ölçeklenir. Bu sayede model, elin kamera karşısındaki konumundan ve ölçeğinden bağımsız hale gelir.

$$\hat{p}_i = \frac{p_i - p_0}{\max_j |p_j - p_0|} \quad (1)$$

burada p_i , i . landmarkın koordinat vektörü ve p_0 bilek koordinatıdır.

Adım 2 — Parmak Ucu Mesafeleri: Beş parmak ucunun (başparmak, işaret, orta, yüzük, serçe: landmark 4, 8, 12, 16, 20) ikili kombinasyonlarından elde edilen 10 Öklid mesafesi özellik vektörüne eklenir. Bu özellikler, parmaklar arası açılma-kapanma ilişkisini sayısal olarak ifade eder.

Sonuç olarak her örnek, $63 + 10 = 73$ boyutlu bir özellik vektörü olarak temsil edilmektedir.

4. Veri Kümesi

4.1. Veri Toplama

Veri toplama işlemi, projeye birlikte geliştirilen özel bir web arayüzü (`collect.html`) aracılığıyla gerçekleştirilmiştir. Bu arayüz; MediaPipe Hands'i tarayıcıda çalıştırmakta, her karede ön işleme adımını uygulamakta ve sonuçları CSV formatında kaydetmektedir. Böylece eğitim sırasındaki ön işleme adımı ile veri toplama sırasındaki ön işleme adımının birebir aynı olması garanti edilmiştir.

4.2. Veri Kümesi İstatistikleri

Toplanan veri kümesi aşağıdaki özelliklere sahiptir:

Tablo 1: Veri kümesi sınıf dağılımı

Sınıf	Örnek Sayısı	Açıklama
open_palm	130	Açık avuç
pinch	138	Kıstırma
fist	118	Yumruk
Toplam	386	

Veri kümesi %80 eğitim, %20 test olarak stratified şekilde bölünmüştür. Stratified bölme, her sınıfın her iki kümede de orantılı olarak temsil edilmesini sağlar.

Tablo 2: Eğitim/Test bölümü

Küme	Örnek Sayısı	Oran
Eğitim	308	%80
Test	78	%20
Toplam	386	

5. Model Mimarileri

Kıyaslamalı bir analiz yapabilmek için üç farklı derin öğrenme mimarisi seçilmiş ve aynı eğitim konfigürasyonu ile eğitilmiştir.

5.1. Çok Katmanlı Algılayıcı (MLP)

MLP, en temel tam bağlı mimariyi temsil etmektedir. 73 boyutlu özellik vektörü doğrudan giriş olarak alınır.

Tablo 3: MLP katman yapısı

Katman	Boyut	Aktivasyon
Giriş	73	—
Tam Bağlı + BN	128	ReLU + Dropout(0.3)
Tam Bağlı + BN	64	ReLU + Dropout(0.3)
Tam Bağlı	32	ReLU
Çıkış	3	(CrossEntropy)

Toplam parametre sayısı: **20,291**

5.2. 1 Boyutlu Evrişimsel Sinir Ağı (1D-CNN)

1D-CNN, 21 landmarkın her birini $\mathbb{R}^3$ uzayında bir zaman adımı gibi işleme fikrine dayanmaktadır. 73 boyutlu düz vektörü önce (21, 3) şeklinde yeniden biçimlendirir, ardından bunu transpoz ederek (3, 21) şeklinde 3 kanallı 1D sinyal haline getirir. Böylece x, y, z koordinatları ayrı kanallar olarak işlenebilmektedir.

Tablo 4: 1D-CNN katman yapısı

Katman	Kanallar	Çekirdek	Aktivasyon
Giriş	3 kanal, 21 nokta	—	—
Conv1d + BN	$3 \rightarrow 64$	3	ReLU
Conv1d + BN	$64 \rightarrow 128$	3	ReLU
AdaptiveAvgPool	$128 \rightarrow 128$	Global	—
Tam Bağlı	$128 \rightarrow 64$	—	ReLU + Dropout(0.3)
Çıkış	$64 \rightarrow 3$	—	(CrossEntropy)

Toplam parametre sayısı: **34,179**

5.3. Uzun Kısa Süreli Bellek (LSTM)

LSTM, 21 landmarkı sıralı bir dizi olarak işleme almaktadır. Her landmark, 3 boyutlu bir giriş vektörü olarak LSTM’e adım adım beslenir.

Tablo 5: LSTM katman yapısı

Katman	Parametre	Detay
LSTM	Giriş: 3, Gizli: 64	2 katman, Bidirectional
Tam Bağlı	$128 \rightarrow 64$	ReLU + Dropout(0.3)
Çıkış	$64 \rightarrow 3$	(CrossEntropy)

Toplam parametre sayısı: **143,107**

5.4. Eğitim Konfigürasyonu

Üç model de aynı hiperparametrelerle eğitilmiştir:

Tablo 6: Eğitim hiperparametreleri

Parametre	Değer
Optimizer	Adam ($lr = 0.001$)
Öğrenme Hızı Planlaması	ReduceLROnPlateau (patience=10, factor=0.5)
Batch Boyutu	32
Epoch Sayısı	100
Kayıp Fonksiyonu	CrossEntropyLoss
Donanım	NVIDIA GPU (CUDA)

6. Deneysel Sonuçlar

6.1. Model Karşılaştırması

Tablo 7: Model performans karşılaştırması

Model	Test Doğruluğu	Parametre	Eğitim Süresi	Durum
MLP	%98.72	20,291	8.9 sn	—
1D-CNN	%100.00	34,179	7.1 sn	En iyi
LSTM	%96.15	143,107	7.8 sn	—

1D-CNN modeli, %100 test doğruluğuyla en iyi performansı sergilemiştir. Üstelik MLP'ye kıyasla yalnızca yaklaşık 14,000 ek parametre gerektirmekte; LSTM'den ise 4 kat daha az parametre kullanmakta ve ondan daha yüksek doğruluk elde etmektedir. Bu durum, 1D-CNN'in el landmark verisi için özellikle uygun bir mimari olduğunu göstermektedir.

6.2. Sınıf Bazlı Performans (1D-CNN)

1D-CNN modelinin test kümesi üzerindeki sınıf bazlı sonuçları aşağıdaki gibidir:

Tablo 8: 1D-CNN sınıf bazlı classification report

Sınıf	Precision	Recall	F1-Score	Destek
fist	1.00	1.00	1.00	24
open_palm	1.00	1.00	1.00	26
pinch	1.00	1.00	1.00	28
Macro Ort.	1.00	1.00	1.00	78

6.3. Neden 1D-CNN Başarılı?

- **Mekansal yerellik:** 1D konvolüsyon, komşu landmarklar arasındaki ilişkileri (örneğin, parmak eklemleri) yerel olarak yakalar.
- **Parametre paylaşımı:** Aynı çekirdek tüm landmark pozisyonlarına uygulanır; bu durum genelleme kapasitesini artırır.
- **Kanal ayrımı:** x, y, z koordinatlarının ayrı kanallar olarak işlenmesi, modelin her eksenini bağımsız olarak öğrenebilmesine olanak tanır.
- **Uygun veri yapısı:** 21 landmarkın her biri sırayla parmak eklemlerine karşılık gelir; bu sıralılık, konvolüsyonel filtrelerin öğrenebileceği anlamlı bir yapı oluşturur.

7. Web Uygulaması

7.1. Mimari Bileşenler

Web uygulaması dört ana JavaScript modülünden oluşmaktadır:

`hand-tracker.js` MediaPipe Hands'i başlatan, kamera aygıtını seçen ve her karede landmark sonuçlarını diğer modüllere ileten sınıf. Akıllı kamera seçim mantığı sayesinde sanal kameralar (örn. telefon yansıtma uygulamaları) atlanarak PC web kamerası tercih edilir.

`gesture-classifier.js` ONNX modeli yükleyen ve landmark verisini sınıflandıran sınıf. ONNX modeli yüklenemezse kural tabanlı bir yedek sistem devreye girer. Çift katmanlı yumuşatma (history tabanlı çoğunluk oyu + stabilite filtresi) ile titreşim önlenir.

`globe-controller.js` Three.js ile 3 boyutlu dünya küresini renderlayan sınıf. Dönme, yakınlaştırma ve kilitleme işlemlerini yönetir.

`ui-overlay.js` Tüm modülleri koordine eden ana sınıf. Gesture sonucuna göre küreyi kontrol eder, arayüz panellerini günceller.

7.2. ONNX Runtime Entegrasyonu

PyTorch'ta eğitilen model, aşağıdaki kodla ONNX formatına aktarılmıştır:

```
1 dummy = torch.randn(1, input_size).to(DEVICE)
2 onnx_path = os.path.join(MODELS_DIR, "cnn_best.onnx")
3 torch.onnx.export(
4     model, dummy, onnx_path,
5     input_names=['input'],
6     output_names=['output'],
7     dynamic_axes={'input': {0: 'batch'}, 'output': {0: 'batch'}}
8 )
```

Listing 1: Model ONNX ihracatı

Tarayıcıda model şöyle yüklenir ve çalıştırılır:

```
1 // Model ykleme
2 this.session = await ort.InferenceSession.create('models/
   cnn_best.onnx');
3
4 // Inference
5 const inputTensor = new ort.Tensor('float32',
6     Float32Array.from(inputArray), [1, 73]);
7 const feeds = {};
8 feeds[this.session.inputNames[0]] = inputTensor;
9 const output = await this.session.run(feeds);
```

Listing 2: ONNX Runtime Web ile model inference

7.3. Gesture – Eylem Eşleştirmesi

Tablo 9: Gesture ve küre eylem eşleştirmesi

Gesture	Açıklama	Eylem
Açık Avuç	Tüm parmaklar açık, el hareket ediyor	Küreyi döndür
Kıstırma	Başparmak + işaret birleşik, el yukarı/aşağı	Yakınlaştır / Uzaklaştır
Yumruk	Tüm parmaklar kapalı	Küreyi kilitle (durdur)
Yok	El algılanmıyor	Oto-dönüşüme devam et

7.4. Kullanılan Teknolojiler

Tablo 10: Proje teknoloji yığını

Teknoloji	Versiyon	Kullanım
<i>Model Eğitimi</i>		
PyTorch	2.x	Derin öğrenme
MediaPipe	0.10.x	El landmark tespiti
scikit-learn	1.x	Metrik hesaplama
ONNX	1.x	Model ihracatı
<i>Web Uygulaması</i>		
Three.js	r128	3B sahne/küre
MediaPipe Hands JS	0.4.x	Tarayıcıda el takibi
ONNX Runtime Web	1.x	Tarayıcıda inference
Vanilla JS/CSS	—	UI/UX

8. Tartışma

8.1. Başarılı Yönler

Proje genel olarak hedeflerini karşılamıştır. 1D-CNN modeli %100 test doğruluğuyla son derece başarılı bir sınıflandırma performansı sergilemiştir. Bunun yanında, model ONNX formatında tarayıcıya aktararak herhangi bir sunucu gerektirmeksizin gerçek zamanlı çalışabilmektedir. Arayüz ise dark-mode uzay temasında estetik ve kullanıcı dostu bir tasarımla tasarlanmıştır.

8.2. Sınırlılıklar

Projenin bazı sınırlılıkları mevcuttur. Her şeyden önce, veri kümesi nispeten küçüktür: 386 örnek, farklı ellere ve aydınlatma koşullarına genellenebilirlik konusunda sorular doğurabilir.

Özellikle etkileşim tarafında, **kıstırma (pinch) hareketi ile zoom yapma algoritmasında donanımsal ve mantıksal zorluklar** yaşanmıştır. Başparmak ve işaret parmağı birleştiğinde oluşan mikro hareketlerin MediaPipe tarafından algılanmasındaki tutarsızlıklar nedeniyle zoom fonksiyonu istenen akıcılıkta çalışmamaktadır. Bu

durum çeşitli filtrelemeler (history, accumulator) ile hafifletilmeye çalışılsa da tamamen çözülememiş ve projenin bir sınırlılığı olarak kalmıştır.

Ek olarak, sekiz ila on bir kare/saniye civarında seyreden düşük FPS değeri, orta düzey donanıma sahip sistemlerde kullanıcı deneyimini olumsuz etkileyebilir. Nihayet, MediaPipe yalnızca tek el algıladığı için iki eli gesture'lar desteklenmemektedir.

8.3. Gelecek Çalışmalar

Bu sınırlılıklar aşağıdaki yönlerde geliştirilebilir:

- Veri artırma teknikleri (rotasyon, gaussian gürültü) ile veri kümesi zenginleştirilebilir.
- Inference işlemi Web Worker'a taşınarak UI bloklanması önlenir.
- Transfer öğrenme ile daha büyük el veri kümelerinden yararlanılabilir.
- İki eli gesture desteği eklenerek etkileşim zenginleştirilebilir.

9. Sonuç

Bu çalışmada, el hareketi tanınması ve 3 boyutlu görüntülemeyi bir arada kullanan uçtan uca bir derin öğrenme sistemi başarıyla geliştirilmiştir. MediaPipe Hands tarafından sağlanan landmark verisinin ön işlenmesi ve 1D-CNN mimarisinin uygulanmasıyla %100 test doğruluğu elde edilmiştir. Eğitilen model, ONNX Runtime Web aracılığıyla doğrudan web tarayıcısında çalıştırılmış; Three.js tabanlı 3 boyutlu dünya küresi ile entegre edilerek interaktif bir uygulama haline getirilmiştir.

Sonuçlar, 1D-CNN'in landmark tabanlı gesture tanıma için hem verimli hem de güçlü bir mimari olduğunu ortaya koymaktadır. Geliştirme sürecinde karşılaşılan kamera seçimi, ön işleme uyumsuzlukları ve gerçek zamanlı smoothing gibi pratik zorluklar da ele alınmış ve çözüme kavuşturulmuştur.

10. Ek Çalışmalar

Bu projede zorunlu gereksinimler ötesinde aşağıdaki ek çalışmalar gerçekleştirilmiştir:

1. **GitHub Repo (Public):** Proje, detaylı Türkçe README, kurulum talimatları ve proje yapısı ile birlikte public GitHub reposunda yayınlanmıştır. <https://github.com/utkumertgecgel/gesture-globe>
2. **Birden Fazla Model Karşılaştırması:** MLP, 1D-CNN ve LSTM olmak üzere üç farklı derin öğrenme mimarisi sistematik olarak karşılaştırılmış; sonuçlar tablo, grafik ve confusion matrix'lerle raporlanmıştır.
3. **ONNX Edge Deployment:** Eğitilen CNN modeli PyTorch'tan ONNX formatına aktarılmış ve ONNX Runtime Web kullanılarak doğrudan tarayıcıda (sunucu gerektirmeden) çalıştırılmıştır. Bu yaklaşım, edge deployment senaryolarına somut bir örnek teşkil etmektedir.
4. **Özel Veri Seti:** Tüm eğitim verisi, proje kapsamında geliştirilen `collect.html` arayüzü aracılığıyla elle toplanmıştır. Harici veri seti kullanılmamıştır.

5. **Canlı Web Demo:** Three.js, MediaPipe Hands ve ONNX Runtime Web entegrasyonu ile çalışan, gerçek zamanlı gesture tanıma ve 3D küre kontrolü sunan bir web uygulaması geliştirilmiştir.
6. **LaTeX Rapor:** Bu rapor, Overleaf kullanılarak LaTeX ile hazırlanmıştır.

Kullanılan Araçlar

Bu proje kapsamında aşağıdaki yapay zekâ araçlarından yardım alınmıştır:

- **Google Gemini / Antigravity:** Kod geliştirme, hata ayıklama ve rapor taslağı oluşturma süreçlerinde yardımcı araç olarak kullanılmıştır. Model tasarımı, veri toplama, eğitim deneyleri ve sonuç analizi tamamen özgün çalışma ürünüdür.

Tüm model mimarisi kararları, hiperparametre seçimleri ve deney sonuçları öğreniciye aittir.

Kaynaklar

- [1] Zhang, F., et al. (2020). *MediaPipe Hands: On-device Real-time Hand Tracking*. arXiv:2006.10214.
- [2] Oudah, M., Al-Naji, A., & Chahl, J. (2020). *Hand Gesture Recognition Based on Computer Vision: A Review of Techniques*. Journal of Imaging, 6(8), 73.
- [3] Kopuklu, O., et al. (2019). *Real-time Hand Gesture Detection and Classification Using Convolutional Neural Networks*. IEEE FG 2019.
- [4] ONNX Community (2023). *ONNX: Open Neural Network Exchange*. <https://onnx.ai>
- [5] Cabello, R. (2023). *Three.js — JavaScript 3D Library*. <https://threejs.org>